

Mô hình hóa Vấn đề / Giải pháp — Lishop

Problem-Solution Modeling

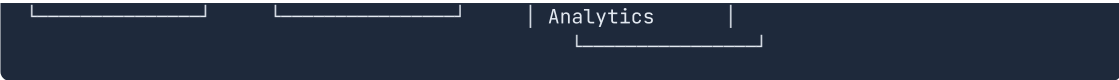
Tài liệu phân tích các vấn đề trong lĩnh vực thương mại điện tử tại Việt Nam và mô hình giải pháp tương ứng mà nền tảng Lishop áp dụng.

1. Mô hình Miền (Domain Model)

1.1 Phân tích Miền Kinh doanh (Business Domain Analysis)

Lishop vận hành trong miền **Marketplace E-commerce** với 6 sub-domain chính:



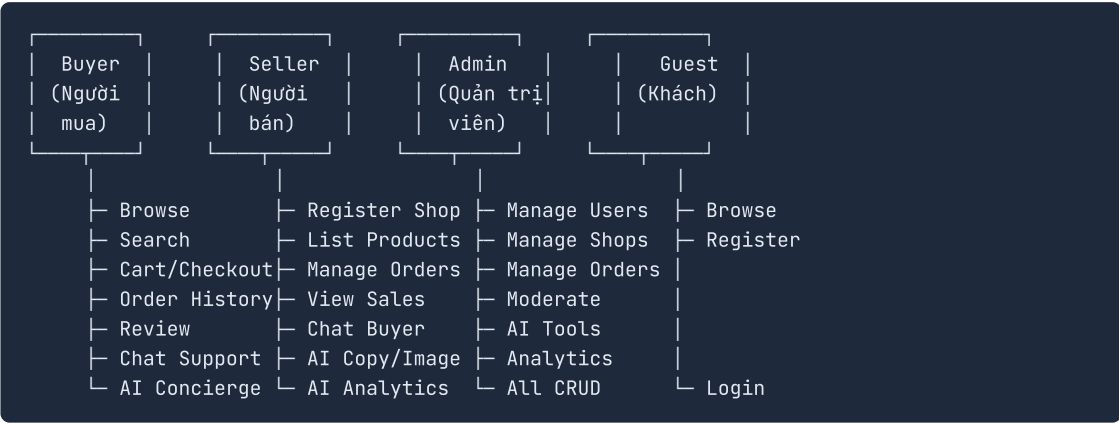


Phân loại Domain:

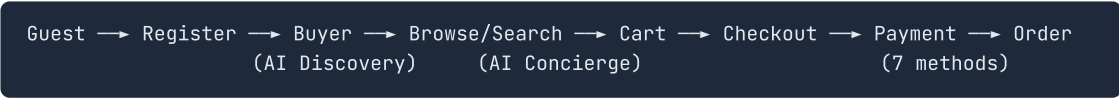
Loại	Domain	Lý do
Core	Catalog, Commerce	Cốt lõi tạo ra giá trị kinh doanh — cho phép mua bán
Supporting	Identity, Logistics, Post-Purchase, Financial	Cần thiết cho vận hành nhưng không phải lợi thế cạnh tranh độc nhất
Generic	Engagement, Notifications	Có thể dùng giải pháp có sẵn, không cần custom sâu
AI	Intelligence	Lợi thế cạnh tranh — khác biệt hóa so với đối thủ

1.2 Mô hình Hành vi (Behavioral Model)

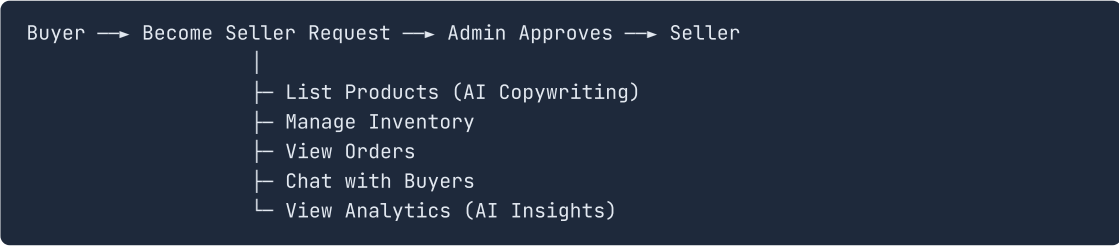
Các tác nhân (Actors) và Use-case chính:



Luồng chính (Main Flow) — Từ Khách đến Đơn hàng:



Luồng phụ (Secondary Flow) — Người bán:



1.3 Mô hình Dữ liệu (Data Model)

25 thực thể chính, phân theo nhóm:

Nhóm	Thực thể	Quan hệ chính
Identity	User , DeviceToken	User 1:N DeviceToken
Catalog	Shop , Category , Product , ProductImage , ProductVariant , Tag , ProductTag	Shop 1:N Product, Product 1:N Variant
Commerce	CartItem , Order , OrderItem , Payment	Order 1:1 Payment
Logistics	Shipment , ShipmentEvent , StockMovement	Shipment 1:N Event
Post-Purchase	Review , ReturnRequest , ReturnItem , Refund , Invoice	Return 1:0..1 Refund
Engagement	Wishlist , Notification , NotificationPreference , LoyaltyPoint	User 1:N Each
Financial	Wallet , WalletTransaction , WalletTopupRequest , Coupon , CouponUsage , FlashSale , FlashSaleItem	Wallet 1:N Transaction

2. Mô hình Vấn đề — Giải pháp (Problem-Solution Mapping)

2.1 Vấn đề Kinh doanh

#	Vấn đề	Bằng chứng (Context)	Giải pháp	Module triển khai
P-01	Phí sàn cao (Shopee/Lazada thu 5-20%)	Project Charter: "các nền tảng phổ biến thu phí cao, kiểm soát dữ liệu người bán"	Xây dựng marketplace độc lập, minh bạch phí	Core platform
P-02	Người bán mất dữ liệu khách hàng	Sàn lớn giữ data, seller không tiếp cận được khách	Seller dashboard: quản lý đơn hàng, sản phẩm, doanh thu	mfe-seller
P-03	Người mua khó tìm sản phẩm	Search từ khóa truyền thống kém hiệu quả với tiếng Việt	AI Product Discovery — tìm kiếm ngôn ngữ tự nhiên	POST /products/ai-discovery
P-04	Phân vân khi mua hàng online	Nhiều lựa chọn,	AI Shopping Concierge	POST /shopping/concierge

#	Vấn đề	Bằng chứng (Context)	Giải pháp	Module triển khai
	(choice paralysis)	không biết mua gì	— gợi ý mua sắm cá nhân hóa	
P-05	Sai size quần áo khi mua online	~30% đơn hàng thời trang bị trả lại vì sai size	AI Style & Fit Advisor — gợi ý size dựa trên số đo	POST /shopping/style-fit-advisor
P-06	Viết mô tả sản phẩm tốn thời gian	Seller mất hàng giờ để viết mô tả cho từng sản phẩm	AI Product Copywriting — tự động sinh mô tả	AdminService.generateProductCopy()
P-07	Nhập liệu sản phẩm hàng loạt khó khăn	Import từ Excel/text dễ sai	AI Import & Enrich — nhập và làm giàu tự động	AdminService.aiImportEnrichProducts()
P-08	Kiểm duyệt đánh giá không theo kịp	Hàng ngàn reviews mới mỗi ngày	AI Review Moderation — kiểm duyệt tự động	POST /admin/reviews/:id/ai-moderation

2.2 Vấn đề Kỹ thuật

#	Vấn đề	Bằng chứng	Giải pháp	Trade-off
T-01	Monolith trở nên khó bảo trì khi quy mô lớn	26 domain modules trong 1 codebase	NestJS Modular Monolith — module hóa chặt chẽ	Không thể deploy độc lập từng module
T-02	Frontend phình to khi thêm tính năng	Một Next.js app sẽ rất lớn	Micro-Frontend với Module Federation v8: 10 MFE apps	10 process chạy đồng thời, tăng độ phức tạp
T-03	Lập code giữa các MFE	Mỗi MFE tự viết UI components	Shared packages: @lishop/ui , @lishop/shared , @lishop/contracts	Phải quản lý version shared packages
T-04	State giữa các MFE không đồng bộ	Auth state, cart count, notification count	@lishop/event-bus — BroadcastChannel-based	Fallback silently fail nếu không hỗ trợ
T-05	Chậm query catalog	Join nhiều bảng category → product → variant	Redis caching: 5min-1hr TTL cho từng loại	Stale data trong cache window

#	Vấn đề	Bằng chứng	Giải pháp	Trade-off
T-06	AI response chậm (2-5 giây)	User phải chờ response	Cache AI response 60-180s tùy feature	Tin cũ trong cache window
T-07	Email gửi chậm, block request	Xác thực email, reset mật khẩu	BullMQ async queue — email qua background worker	Thêm Redis dependency
T-08	Xử lý 7 payment gateways phức tạp	Mỗi gateway khác nhau (signature, callback)	PaymentsGateway — abstraction layer	Abstraction leak nếu gateway quá khác biệt
T-09	Real-time cần 2 chiều	Notification 1 chiều + Socket.IO 2 chiều	SSE (notification stream) + Socket.IO (realtime gateway)	2 hệ thống real-time cần maintain

2.3 Vấn đề Thị trường Việt Nam

#	Vấn đề	Bằng chứng	Giải pháp	Chi tiết
V-01	70% người Việt dùng COD (trả tiền khi nhận hàng)	Thói quen thanh toán	COD payment method + admin confirmation flow	<code>PaymentMethod.COD</code> , <code>PATCH /admin/payments/:orderId/confirm</code>
V-02	VNPAY là gateway phổ biến nhất	90%+ ngân hàng Việt	VNPAY integration với HMAC-SHA512	<code>PayVNPayService</code> , sandbox DEMO mode
V-03	MoMo là ví điện tử số 1	30M+ người dùng	MoMo IPN callback	Server-to-server, auto-complete khi nhận IPN
V-04	ZaloPay qua Zalo messaging	Zalo là messaging app lớn nhất VN	ZaloPay callback-only, user ở lại ZaloPay app	Không redirect, chỉ server-to-server
V-05	Chuyển khoản ngân hàng cho số tiền lớn	Người Việt tin tưởng chuyển khoản	Wallet + manual top-up (admin xác nhận)	<code>WalletTopupRequest</code> PENDING → APPROVED
V-06	GHN/GHTK/Viettel Post là 3 hãng vận chuyển chính	thị phần vận chuyển Việt Nam	Abstraction: <code>GET /shipping/rates?cityName&weightGrams</code>	3 carrier tích hợp qua 1 endpoint
V-07	Tiếng Việt có dấu, tìm kiếm khó	Từ khóa tiếng Việt phức tạp	<code>normalizeText()</code> — bỏ dấu, mapping đ → d	AI discovery xử lý ngữ nghĩa tiếng Việt
V-08	Thuế VAT 10% bắt buộc	Hóa đơn VAT cho doanh nghiệp	Invoice model: <code>vatPercent</code> , <code>vatVnd</code>	Tự động tính VAT trên subtotal

#	Vấn đề	Bằng chứng	Giải pháp	Chi tiết
V-09	Người Việt thích tích điểm/loyalty	Văn hóa tích điểm	LoyaltyPoint : 1 point / 1,000 VND	Tích điểm từ đơn hàng
V-10	OAuth: Google + Facebook chiếm ~95%	Google ~90%, Facebook 70M+ users	Google OAuth + Facebook OAuth	Social login, zero-password onboarding

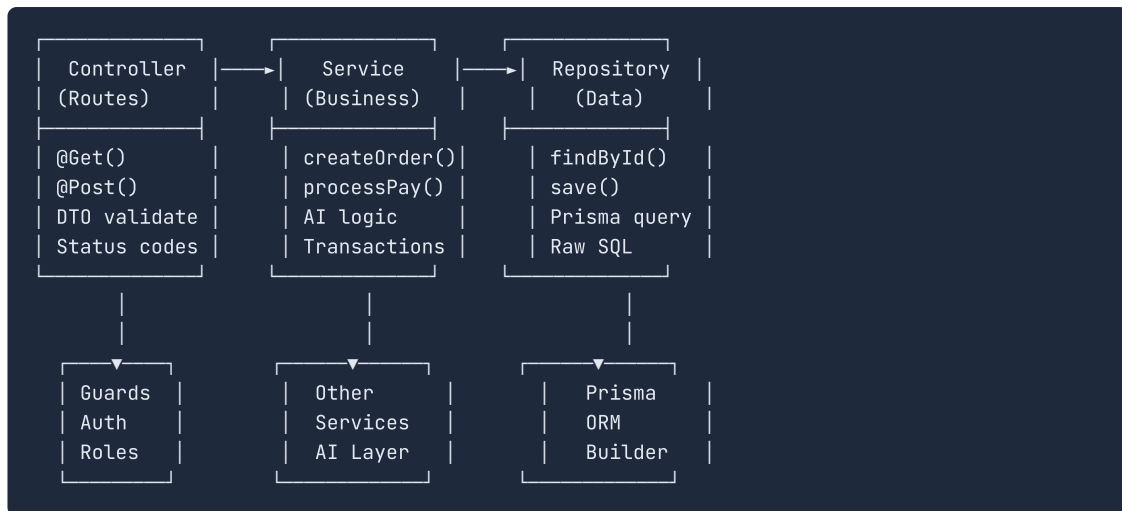
3. Mô hình Kiến trúc (Architecture Model)

3.1 Mô hình Tổng quan

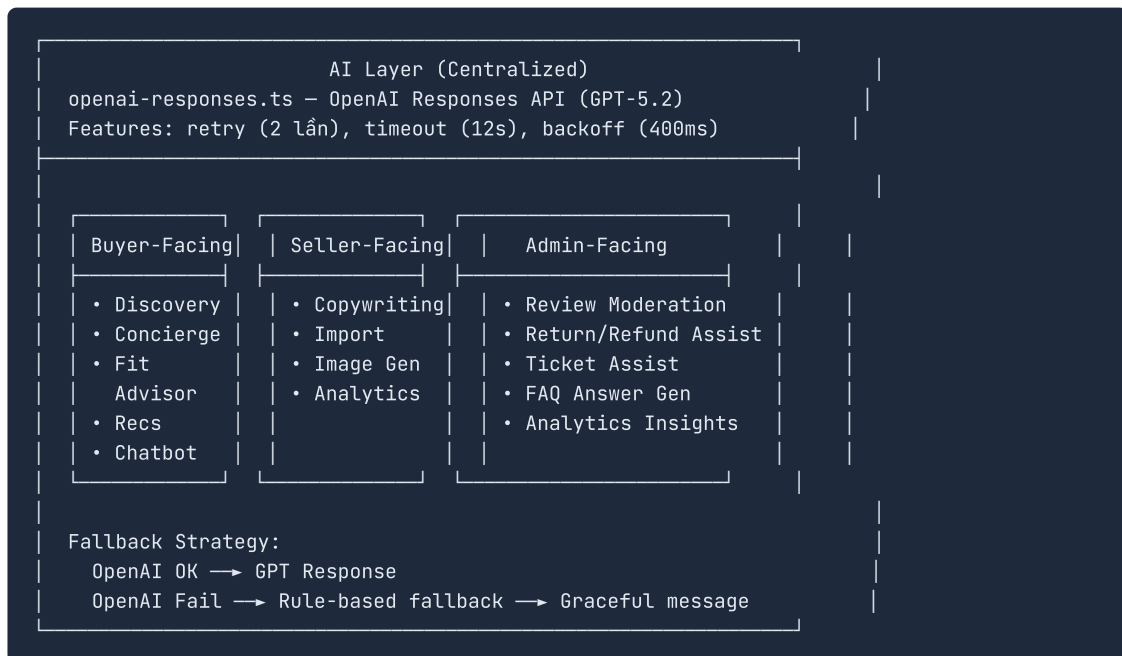


3.2 Mô hình Module Backend (3-Layer Pattern)

Mỗi module tuân theo mô hình 3 lớp nghiêm ngặt:



3.3 Mô hình AI Layer



4. Mô hình Quyết định Kiến trúc (Architecture Decision Records)

ADR-01: Modular Monolith vs Microservices

Context: Dự án có 26 domain module, team 5-6 devs.

Decision: NestJS Modular Monolith.

Lý do:

- Team nhỏ không đủ resources vận hành microservices (monitoring, deployment, debugging)
- 26 modules đều chia sẻ database transaction — microservices gây phức tạp cho distributed transactions

- Module boundaries được enforce bằng NestJS module system — dễ dàng tách thành service riêng sau này nếu cần

Hệ quả:

- ☒ Deploy đơn giản: 1 Docker image
- ☒ Debug dễ dàng: 1 process, 1 log stream
- ☒ Không thể scale module riêng lẻ
- ☒ Build time tăng dần theo modules

ADR-02: Micro-Frontend vs SPA

Context: 11 frontend apps, features phát triển song song.

Decision: Module Federation v8 với 10 MFE remotes + 1 shell host.

Lý do:

- Team phát triển độc lập: mỗi MFE có thể dev/build/test riêng
- Giảm build time: thay đổi 1 MFE không cần rebuild toàn bộ
- Tăng tốc development: hot reload trong từng MFE

Hệ quả:

- ☒ Independent development và deployment
- ☒ Cô lập lỗi: 1 MFE crash không ảnh hưởng MFE khác
- ☒ 10 Node.js processes chạy đồng thời (RAM ~500MB+)
- ☒ Phức tạp routing: Nginx phải map path cho từng MFE
- ☒ Cross-MFE state cần event bus

ADR-03: Fastify vs Express

Context: NestJS hỗ trợ cả 2 HTTP adapters.

Decision: Fastify.

Lý do:

- 2x throughput so với Express (benchmark)
- Built-in schema validation
- TypeScript-first design

Hệ quả:

- ☒ API throughput cao hơn
- ☒ Ecosystem nhỏ hơn Express
- ☒ Một số NestJS packages chỉ hỗ trợ Express

ADR-04: Prisma vs TypeORM vs Drizzle

Context: Cần ORM cho PostgreSQL 16.

Decision: Prisma 5.

Lý do:

- Type safety mạnh nhất — auto-generated types từ schema
- Migration system tốt nhất — detect changes, generate SQL

- Developer experience: Prisma Studio, auto-completion

Hệ quả:

-  Type safety: frontend cũng dùng contracts từ schema
-  Migration an toàn với production DB
-  Query performance chậm hơn raw SQL 10-30% (mitigated by Redis caching)
-  Không support được tất cả PostgreSQL features

ADR-05: OpenAI Responses API vs Chat Completions

Context: Cần AI cho 14 features.

Decision: OpenAI Responses API (GPT-5.2), `/v1/responses` .

Lý do:

- Cấu trúc output tốt hơn với structured outputs
- Future-proof: OpenAI đang migrate từ Chat Completions

Hệ quả:

-  Structured outputs dễ parse
-  Ít tài liệu và code examples hơn

ADR-06: 7 Payment Gateways

Context: Thị trường Việt Nam cần đa dạng payment methods.

Decision: Tích hợp 7 gateways: VNPAY, MoMo, ZaloPay, Stripe, PayPal, Wallet, COD.

Lý do:

- VNPAY + MoMo + ZaloPay: >95% thị trường payment Việt Nam
- Stripe + PayPal: cho khách quốc tế
- Wallet + COD: tăng conversion (người Việt thích COD)

Hệ quả:

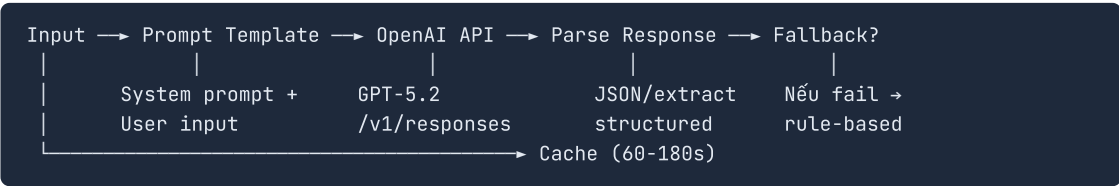
-  Tối đa conversion rate
-  Đáp ứng mọi thói quen thanh toán
-  7 integrations cần maintain và test

5. Mô hình AI — Chi tiết 14 Features

Phân loại theo Actor

Buyer (5 features)	Seller (3 features)	Admin (6 features)
Product Discovery	Product Copywriting	Review Moderation
Shopping Concierge	Import & Enrich	Return Assist
Style & Fit Advisor	Image Generation	Refund Assist
Personalized Recs	Analytics Insights	Ticket Assist
Support Chatbot		FAQ Answer Gen
		Analytics Insights

Mô hình Xử lý AI Chung



Chi tiết từng Feature

Feature	Input	Output	Cache TTL	Fallback
Product Discovery	Query + catalog candidates	Vietnamese text + products	120s	Keyword search
Shopping Concierge	Message + cart + catalog	JSON: cartPlan, reply	90s	Static suggestions
Style & Fit Advisor	Height, weight, gender + variants	JSON: variantId, confidence	180s	Heuristic sizing chart
Recommendations	Wishlist + history + related	Ordered product IDs	300s	Return candidates as-is
Chatbot	Message + context (FAQ, orders)	Vietnamese text + actions	60s	Canned keyword responses
Copywriting	Name + category + details	Vietnamese description	N/A	Template-based
Import & Enrich	Raw text/CSV	Structured products	N/A	Basic CSV parser
Image Gen	Product name + category	Image URL	N/A	Default placeholder
Review Moderation	Review content	APPROVE/REJECT + reason	N/A	Always APPROVE
Return Assist	Return reason + order	Decision + admin note	N/A	Forward to admin
Refund Assist	Refund reason + payment	Action + notes	N/A	Forward to admin
Ticket Assist	Conversation history	Summary + draft reply	N/A	Empty reply
FAQ Answer	Question	Vietnamese answer	N/A	Empty
Analytics Insights	Revenue + order + product data	Highlights + risks + actions	600s	Basic stat summary

6. Mô hình Rủi ro (Risk Model)

Rủi ro Kỹ thuật

Rủi ro	Xác suất	Tác động	Mitigation
OpenAI API downtime ảnh hưởng 14 features	Thấp	Cao	Fallback cho mọi feature, caching
Payment gateway thay đổi API	Trung bình	Cao	Abstraction layer, test định kỳ
Prisma migration conflict trên production	Thấp	Cao	Staging environment, rollback script
Module Federation shared dep version mismatch	Trung bình	Trung bình	Lock versions trong shared config
Redis memory full (caching + queue)	Thấp	Trung bình	TTL, maxmemory policy, monitoring

Rủi ro Kinh doanh

Rủi ro	Xác suất	Tác động	Mitigation
Chi phí OpenAI vượt ngân sách	Thấp	Cao	Caching, rate limiting, fallback
Người dùng không adopt AI features	Trung bình	Cao	UX tối ưu, onboarding guides
Seller adoption thấp	Trung bình	Cao	Onboarding flow đơn giản, hỗ trợ AI
Bảo mật thanh toán bị khai thác	Thấp	Rất cao	httpOnly cookies, rate limiting, Helmet

7. Mô hình Tăng trưởng (Growth Model)

Scaling Strategy

```
Phase 1 (Hiện tại): 1 server, Docker Compose
|
Phase 2 (10K users): Tách API server + DB riêng
|
Phase 3 (50K users): Read replicas, Redis Cluster
|
Phase 4 (100K+): Có thể tách hot modules thành services riêng
                  (payments, AI, notifications)
```

Bottleneck dự kiến

Bottleneck	Ngưỡng	Mitigation
Database connections	~200 concurrent	Connection pooling, read replicas
Redis memory	~4GB	TTL, eviction policy
AI API rate limits	~500 req/min	Caching, queue, fallback
BullMQ job processing	~1000 jobs/min	Worker scaling
Nginx connections	~1024	Increase worker_connections

8. Tổng kết (Summary)

Lishop giải quyết **3 lớp vấn đề**:

1. **Kinh doanh** — Marketplace độc lập, phí thấp, AI-powered, data minh bạch
2. **Kỹ thuật** — Modular monolith + micro-frontends, cân bằng giữa đơn giản và scalable
3. **Thị trường Việt Nam** — 7 payment methods, 3 carriers, tiếng Việt, VAT, loyalty

Mô hình kiến trúc lựa chọn là **pragmatic scalability**:

- Không over-engineer với microservices ngay từ đầu
 - Module boundaries rõ ràng cho phép tách rời sau này
 - AI là lợi thế cạnh tranh cốt lõi, tích hợp sâu vào mọi domain
 - Local adaptation (VNPAY, MoMo, GHN, tiếng Việt) là barrier to entry
-

Tài liệu được tạo ngày 26/06/2026. Dựa trên phân tích mã nguồn và tài liệu dự án Lishop.